

Midterm Project Report

Measuring the Recall Cost of Approximate Nearest Neighbor Search: A Parameter Selection Heuristic for Vector Databases

EMMANUEL C. ENENTA

Senior Seminar · Fisk University · Weeks 1 to 7

1. Project Overview

Title. Measuring the Recall Cost of Approximate Nearest Neighbor Search: A Parameter Selection Heuristic for Vector Databases.

Problem. Semantic search, recommendation and retrieval-augmented generation all rely on approximate nearest neighbor (ANN) indexes, which trade some accuracy for speed through tunable settings. Developers usually leave those settings at library defaults, and because an approximate index still returns plausible results when it misses the true neighbors, the lost recall produces no error and no warning.

Goals. Measure how much recall library defaults give up across index types and datasets; map the best recall achievable at each latency; build and test a model that predicts the setting a dataset needs from cheap properties of the data instead of an exhaustive sweep; and make it all reproducible.

Expected outcome. A final research report, a public code repository with the measurement harness and every raw result, the prediction model with a held-out evaluation, and a final presentation.

1.1 How the methods work

Recall and latency. For every query, exact brute-force search gives the true 10 nearest neighbors. Recall@10 is the share of those 10 that the index actually returns, averaged over all queries: 1.0 is perfect, and 0.335 means about two of every three true neighbors were missed. Latency is the time per query on one CPU thread. Every setting studied here trades one for the other.

IVF (inverted file index). When the index is built, k-means splits the collection into $nlist$ cells (100 by default, 1,024 in my main sweeps), each with a center, and every vector is filed under its nearest center. To answer a query, the index compares it with all the centers and scans only the vectors in the $nprobe$ closest cells. A true neighbor is missed when it sits just across a cell border, in a cell that was not scanned (Figure 1, left). Raising $nprobe$ scans more cells, so recall rises and the time grows roughly in step with the number of vectors scanned. FAISS uses $nprobe = 1$ by default. IVF-Flat keeps the full vectors; IVF-PQ compresses each one into a short code, 8 bytes at the default settings (product quantization), which saves memory but makes every distance approximate, so even scanning every cell cannot reach full recall.

HNSW (hierarchical navigable small world graph). Every vector becomes a node linked to about $M = 16$ of its near neighbors. A random few are also placed in sparser upper layers with longer links, like express lanes. A search enters at the top layer, keeps stepping to whichever neighbor is closer to the query, drops a layer when it cannot get closer, and repeats down to the bottom layer, which holds every vector (Figure 1, right). There it keeps a list of the ef best candi-

dates found so far and keeps expanding them until none of their neighbors is closer. With a small ef the walk is fast but greedy and can settle in the wrong neighborhood; a larger ef explores more of the graph, which raises recall and costs time. `hnswlib`'s default is $ef = 10$, and it silently raises ef to k whenever k is larger.

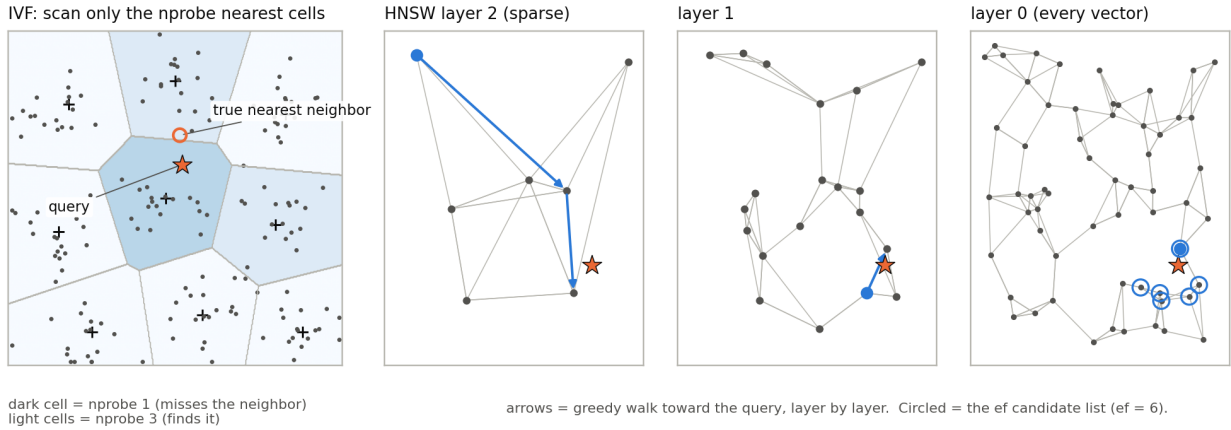


Figure 1. How each index answers a query (toy 2-D data, for illustration). Left: IVF scans only the $nprobe$ cells nearest the query, so a neighbor just across a border is missed at $nprobe = 1$. Right: HNSW walks greedily down a stack of graphs, then keeps ef candidates on the bottom layer.

The trade-off in numbers. Table 1 shows both settings on SIFT. Moving HNSW from the default $ef = 10$ to $ef = 48$ lifts recall from 0.729 to 0.961 for about three times the query time. Moving IVF-Flat from $nprobe = 1$ to $nprobe = 32$ lifts it from 0.308 to 0.967 for about twelve times the time. Past that point each extra bit of recall gets expensive, so the useful question is the smallest setting that reaches a target, which this project calls the oracle setting.

index	setting	recall@10	ms per query	note
HNSW	$ef = 10$	0.729	0.041	library default
HNSW	$ef = 48$	0.961	0.119	oracle for 0.95
HNSW	$ef = 1024$	1.000	1.727	
IVF-Flat	$nprobe = 1$	0.308	0.030	library default
IVF-Flat	$nprobe = 32$	0.967	0.365	oracle for 0.95
IVF-Flat	$nprobe = 256$	1.000	2.586	

Table 1. The speed and accuracy trade-off on SIFT (200,000 vectors, IVF with 1,024 cells, one search thread).

Local intrinsic dimensionality (LID). Vectors are stored in 25 to 784 dimensions, but real data rarely uses all of them; handwritten digits vary along far fewer directions than their 784 pixels. LID estimates how many dimensions the data effectively has around one point. I compute it from the distances to that point's 20 nearest neighbors with the Levina-Bickel maximum-likelihood estimator, $LID = -1 / \text{mean}(\ln(r_1 / r_{20}))$ over $i = 1$ to 19. If the 20th neighbor is much farther away than the 1st, the space is locally low-dimensional; if all 20 sit at nearly the same distance, it is high-dimensional. High LID hurts both index types for the same reason: the true neighbors are barely closer than many near misses, so they scatter over more IVF cells and the graph walk has more ways to turn the wrong way. That is why the setting needed for a fixed recall grows with LID, and why LID is the main input to my model.

Hubness. Hubness measures how unevenly points appear in other points' 10-nearest-neighbor lists. In high-dimensional data a few hub points show up in many lists while most points appear in almost none; I report the skew of those counts. Week 7 tests it as a second feature.

2. Work Completed: Weeks 1 to 7

Over seven weeks the project grew to 2,003 lines of Python, 12 datasets and 4,101 timed measurements. The work fell into four phases.

Weeks 1 and 2: topic and proposal

I narrowed a broad interest in vector search to one measurable question, chose the ANN-Benchmarks datasets because they ship with published ground truth, and wrote the proposal with explicit success criteria: the harness must reproduce published results, the study must cover every index and dataset pair, and the model must be judged on data it has not seen.

Weeks 3 and 4: measurement harness and first study

I built the harness in Python with FAISS (IVF-Flat, IVF-PQ), hnswlib (HNSW), NumPy, scikit-learn and pandas: data loading, exact ground truth, index wrappers, dataset characterization, parameter sweeps and Pareto-frontier analysis. Checking my exact search against the published neighbors gave 0.9991 agreement on SIFT but only 0.9856 on NYTimes; all 102 NYTimes disagreements were exact distance ties between duplicate vectors, so I defined recall on distances, which brought agreement to 1.0000 everywhere. I then ran 657 timed measurements over 219 configurations on SIFT, GloVe-100 and NYTimes. The nine library defaults averaged a recall@10 of 0.455.

Weeks 5 and 6: more data and the prediction model

I added Fashion-MNIST, MNIST and GloVe-25, plus six synthetic datasets with a known intrinsic dimension (4 to 48), which also let me check the dimensionality estimator: it reads 4.3 for a true 4 and 7.6 for a true 8, but only 22.8 for a true 48. Fine-grained sweeps found, for each dataset, the cheapest setting that reaches a target recall. I built the model (log2 of the needed setting as a linear function of local intrinsic dimensionality, LID) and tested it leave-one-dataset-out against the library default, fixed settings and the oracle. I also ran the defaults at $k = 1$ and $k = 100$ and on the full million-vector collections, added 95% confidence intervals, and measured how much recall changes when the same index is rebuilt. This phase added 2,556 measurements.

Week 7: collection size and hubness

The weeks 5 and 6 results suggested collection size was tangled up with LID, so I swept SIFT and GloVe-100 at 50,000, 100,000, 500,000 and full size (200,000 already existed), 888 more measurements. I also added hubness, the skew of how often each point appears in other points' neighbor lists, as a feature for all twelve datasets, extended the model to several features, and repeated the held-out test with every size of the held-out dataset removed from training.

3. Evidence of Progress

Repository. All code, raw measurements, figures and both demos:

github.com/chinedu-2002/ann-recall-study. Running `python scripts/predict_demo.py` generates an unseen dataset, predicts its setting and checks the prediction live.

3.1 The default gap on six datasets

With six datasets the default gap depends on the index type. IVF-Flat leaves recall on the table everywhere: at the default's own latency a tuned index gains between 0.171 and 0.291 recall on every dataset. The HNSW default falls as LID rises, from 0.93 on MNIST and Fashion-MNIST to 0.515 on GloVe-100 (Figure 2).

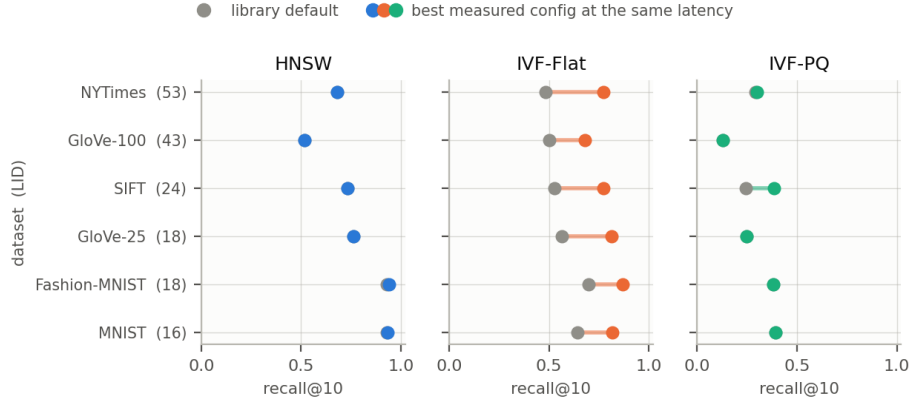


Figure 2. Default recall (gray) against the best measured configuration at the same latency (colored), $k = 10$. Rows are sorted by LID, shown in parentheses.

3.2 The prediction model

Across all twelve datasets, the log of the smallest setting that reaches recall 0.95 tracks query-point LID with a correlation of 0.963 for HNSW and 0.959 for IVF-Flat (Figure 3). Synthetic and real datasets fall on the same line.

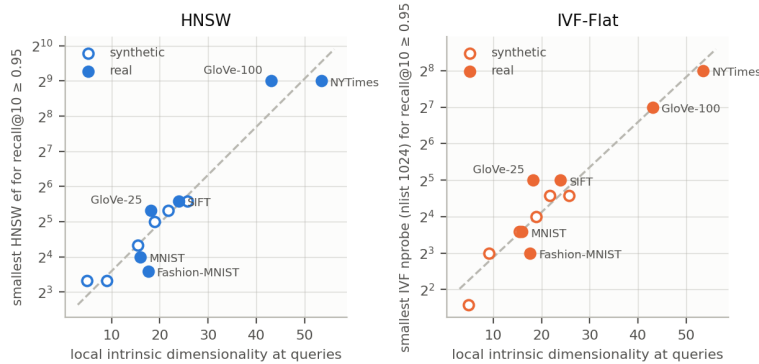


Figure 3. Smallest setting reaching recall@10 of 0.95 against LID for all twelve datasets. Filled points are real, hollow points synthetic; the dashed line is the fitted model.

3.3 Held-out evaluation, collection size and hubness

Each dataset is held out in turn: the model is fitted on the other eleven (every size of the held-out dataset removed) and must predict the held-out dataset's setting. In weeks 5 and 6 the library default never reached recall 0.95 on a held-out dataset, and the only fixed setting that always did cost 4.79 times the oracle's latency on HNSW. Week 7 asked whether collection size and hubness improve the model.

On HNSW, size matters on its own (Figure 4). From 50,000 vectors to the full collection, the default's recall falls from 0.805 to 0.689 on SIFT and from 0.562 to 0.464 on GloVe-100, and the ef

needed for 0.95 doubles on SIFT (32 to 64) and quadruples on GloVe-100 (256 to 1,024). The fitted size term says the needed ef grows by a factor of 1.31 each time the collection doubles. Adding it cuts the held-out HNSW error from 0.79 to 0.58 log2 units and the margin version's cost from 1.55 to 1.30 times the oracle at the same five-of-six hit rate (Table 2). It also fixes the MNIST over-prediction: the predicted ef drops from 24 to 16, exactly the true value. For IVF-Flat with a fixed list count the size term is near zero (-0.08) and does not help. Hubness confirms GloVe-100 is unusual (skew 5.25, against at most 2.84 elsewhere), but adding it barely moves the error (0.58 to 0.55) and GloVe-100 is still under-predicted (256 against a true 512).

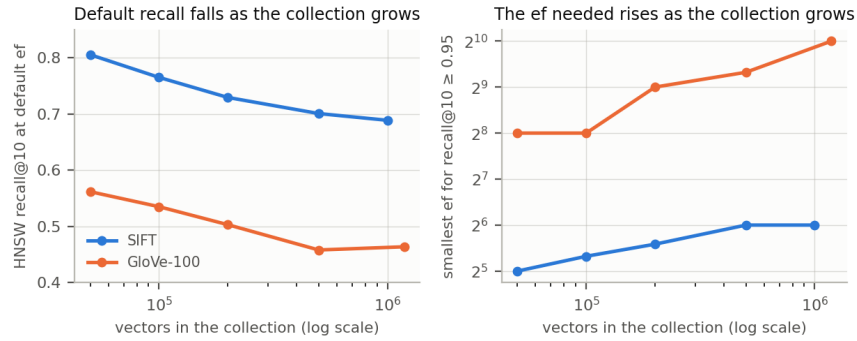


Figure 4. SIFT and GloVe-100 at five collection sizes. Left: HNSW recall at the default ef. Right: the smallest ef reaching recall 0.95 (log scale).

model features	HNSW error (log2)	HNSW + margin: hits, cost	IVF-Flat error (log2)	IVF-Flat + margin: hits, cost
none (one fixed setting)	2.25	4 / 6, 2.73×	1.86	4 / 6, 2.19×
LID	0.79	5 / 6, 1.55×	0.50	5 / 6, 1.48×
LID + size	0.58	5 / 6, 1.30×	0.60	5 / 6, 1.54×
LID + size + hubness	0.55	5 / 6, 1.19×	0.67	5 / 6, 1.47×

Table 2. Held-out test, target recall@10 = 0.95. Error is the mean distance between the predicted and true setting in log2 units; 0.58 means within a factor of about 1.5.

3.4 Code example

The core metric, tie-aware recall, counts a returned neighbor as correct if it is at least as close as the true k-th neighbor, so duplicate vectors are not penalized (src/sweep.py):

```
def per_query_tieaware(found_d, truth_d, k, rtol=1e-5):
    thresh = truth_d[:, k - 1:k] # distance of the true k-th neighbor
    tol = rtol * np.maximum(1.0, np.abs(thresh)) # floating-point tolerance
    return (found_d[:, :k] <= thresh + tol).sum(axis=1) / k
```

4. Progress Compared with the Original Proposal

The proposal scheduled the harness for weeks 1 to 3, the parameter sweeps for weeks 4 to 8, the heuristic for weeks 9 to 11 and the write-up for weeks 12 to 14. Table 3 compares that plan with what happened. The project is ahead of schedule: the sweeps are finished and the model's first version arrived about four weeks early.

planned task	planned	actual status
Measurement harness and baseline check	weeks 1 to 3	Done in weeks 3 and 4. Baseline check changed to published ground truth (see below).
Parameter sweeps, three index types on three datasets	weeks 4 to 8	Done and extended by week 7: six real and six synthetic datasets, $k = 1$ and 100, full collections, five collection sizes.
Fit and validate the heuristic on held-out data	weeks 9 to 11	First version done in weeks 5 and 6, size and hubness added in week 7. In progress: build parameters, error bars.
Analysis, final report, presentation	weeks 12 to 14	Not started , but the analysis and report pipeline already exists from the progress reports.

Table 3. Original timeline against actual progress.

Changes to the plan and why.

Baseline check. The proposal planned to reproduce a published recall-latency curve. Latency depends on the hardware, so I used the published ground truth instead, which tests correctness exactly on every dataset.

Twelve datasets instead of three. A held-out test on three datasets would fit a line through two points. Synthetic data adds datasets whose intrinsic dimension is known.

Stronger evaluation and new sub-studies. Leave-one-dataset-out with explicit baselines replaced a single held-out dataset. Rebuild variance, result-set size and collection size were added because results raised them.

5. Challenges and Solutions

challenge	effect on the project	what I did	status
Exact search disagreed with published neighbors on NYTimes (0.9856)	Recall would have been understated	Traced all 102 cases to distance ties; measured recall on distances	Resolved
hnsplib silently raises ef to at least k	First sweep showed a plateau that did not exist	Grid respects the clamp; confirmed directly at $k = 100$	Resolved, now a finding
Intrinsic-dimension estimator returned 7.5×10^8 on NYTimes	Main heuristic feature unusable	Drop zero distances from duplicates, trimmed mean; validated on synthetic data	Resolved
Rebuilding the same index changes recall (NYTimes 0.590 to 0.679)	One build understates uncertainty	Measured by thread count and insertion order; ranges reported	Open: error bars planned
Collection size tangled up with LID	MNIST over-predicted	Week 7 size sweep; size term added	Resolved for HNSW
Estimator saturates; synthetic LID tops out near 26	High end of the model rests on two datasets	Flagged in every report	Open

Table 4. Major challenges from weeks 1 to 7.

6. Current Project Status

The measurement side is complete: 4,101 validated measurements cover twelve datasets, three index types, three result-set sizes and collections from 50,000 to 1,183,514 vectors, and every table and figure regenerates from committed data. The LID and size model predicts the needed setting within a factor of about 1.5 on unseen datasets and, with a margin, reaches the target on

five of six at 1.30 times the oracle's latency. Still incomplete: the GloVe misses are unexplained, the model covers query settings but not build settings, oracle settings have no error bars yet, and the final report and presentation are not written.

7. Plan for the Second Half of the Semester

weeks	tasks	milestone
8	Rebuild every oracle setting three times with shuffled insertion order; refit the LID and size model with those error bars.	Oracle settings with uncertainty
9 and 10	Extend the model to build settings (HNSW M, IVF list count) and their memory cost; test other features for word embeddings; make the synthetic generator reach LID 40 to 55.	Model covers build and query settings
11	Final held-out evaluation, including full-size collections; freeze all results.	Final results
12 and 13	Write the final report: methods, results, limitations.	Report draft, then final
14	Final presentation and repository clean-up with a one-command reproduction script.	Presentation delivered

Table 5. Plan for weeks 8 to 14.